

SERVICE ACTUEL · KEYCLOAK

Authentification OIDC et configuration du realm

6 capacités analysées

services/keycloak · Keycloak 26.7.4 dans une image JVM, PostgreSQL, opérateur Keycloak Kubernetes, thème et plugin JAR ; Helm, scripts shell/jq et keycloak-config-cli pour la configuration.

Fournir connexion, tokens, sessions et page de login, puis réconcilier la configuration d'authentification livrée avec la plateforme.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : IAM, Notifications, Frontends, Audit.

Périmètre de lecture

- Rapport identité-gouvernance existant ; Dockerfile ; clients OIDC ; définition du realm et flows.
- CR Keycloak, Job realm-sync, CronJob de drift et scripts d'invariants. Aucun realm de client interrogé.

Objets et cycle de vie

- Le realm contient clients, scopes, rôles, flows, sessions et politiques ; IAM garde les identités et droits métier.
- Le chart gère seulement les objets explicitement déclarés et certains invariants ; les fournisseurs fédérés et objets propres à une installation ne sont pas tous possédés par le chart.

Dépendances du service

- PostgreSQL pour Keycloak ; opérateur Kubernetes via la CR k8s.keycloak.org/v2alpha1.
- JAR de thème et plugin ; keycloak-config-cli pour import ; IAM pour clients dynamiques et délégations.

Fonctionnalités et compatibilité à préserver

KC-01 Cible non qualifiée

Connecter les interfaces Web par authorization code et PKCE

COMPORTEMENT OBSERVÉ

Le chart déclare le client public de frontend avec standardFlowEnabled, désactive implicitFlow et exige PKCE S256 côté serveur. Les origines et redirections viennent du rendu de la configuration. Le password grant est un opt-in de test partagé ; il n'est pas implicitement activé par le mode développement. Les scopes incluent identité et audience nécessaire au contrôle de la gateway.

UTILISATEURS ET RÔLES

Utilisateurs des interfaces Web.

INTERFACES DANS LE CODE

Template realm_clients/_api_client_front.tpl ; endpoints OIDC du realm fournis par Keycloak.

DONNÉES ET ÉTATS

Realm, client frontend, scopes, audiences et sessions Keycloak.

DÉPENDANCES ET ÉCHANGES

Frontend, IAM et gateway APISIX.

COMPORTEMENT CIBLE PROPOSÉ

Conserver authorization code avec PKCE, validation d'audience et logout dans un fournisseur sans JVM. IAM Rust garde les identités métier, et les interfaces TypeScript consomment le protocole OIDC sans dépendre de claims propriétaires non documentés.

MIGRATION DE CETTE CAPACITÉ

Inventorier redirections et claims contractuels, configurer le nouvel issuer puis effectuer une période bornée de double validation. Réauthentifier les utilisateurs à la coupure ; ne pas importer aveuglément des sessions.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le nouveau login refuse une redirection non déclarée et un code sans vérificateur PKCE. Les appels API ont la bonne audience et les mêmes profils métier après connexion.

Références de l'analyse

[monorepo · services/keycloak/chart/templates/realm_clients/_api_client_front.tpl:50](#) Flux navigateur déclaré.

[monorepo · services/keycloak/chart/templates/realm_clients/_api_client_front.tpl:105](#) PKCE obligatoire côté IdP.

KC-02 **Cible non qualifiée****Authentifier CLI et SDK par device authorization****COMPORTEMENT OBSERVÉ**

Le client public cleyrop-cli utilise le device grant, sans redirection ni origine Web et sans standard ou implicit flow. Il délivre des refresh tokens selon les politiques du realm et propose offline_access en scope optionnel. Le PKCE du device grant est piloté par un paramètre dédié ; le template documente que le SDK livré n'envoie pas encore de challenge. Le password grant reste lié à l'opt-in de test.

UTILISATEURS ET RÔLES

Utilisateurs CLI, SDK Python et outils interactifs.

INTERFACES DANS LE CODE

Template realm_clients/_cli.tpl ; device authorization et token endpoints du fournisseur.

DONNÉES ET ÉTATS

Client CLI, demande device, session utilisateur et refresh tokens.

DÉPENDANCES ET ÉCHANGES

SDK, IdP et API de la plateforme.

COMPORTEMENT CIBLE PROPOSÉ

Qualifier device grant, polling, expiration et stockage des credentials avec le fournisseur sans JVM. Préserver le lien entre SDK et session utilisateur, en ajoutant PKCE seulement avec une version de client compatible.

MIGRATION DE CETTE CAPACITÉ

Tester l'ancien et le nouveau SDK contre le fournisseur choisi. Versionner le changement PKCE et invalider les anciens refresh tokens lors du retrait d'issuer.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une CLI peut terminer le login sur un autre appareil, puis renouveler l'accès. Une demande device expirée échoue et la révocation de session rend le renouvellement impossible.

Références de l'analyse

[monorepo](#) · [services/keycloak/chart/templates/realm_clients/_cli.tpl:13](#) Absence de flux navigateur.

[monorepo](#) · [services/keycloak/chart/templates/realm_clients/_cli.tpl:50](#) Bascule PKCE et device grant.

KC-03 **Cible non qualifiée****Fournir des clients confidentiels et des audiences de service****COMPOTEMENT OBSERVÉ**

Le client API est confidentiel, active les service accounts et déclare scopes et protocol mappers. Le chart définit aussi les clients nécessaires aux intégrations retenues. Les invariants ajoutent l'audience gateway aux clients non déclarés qui doivent appeler les API et le scope profile aux comptes techniques. La configuration de délégation MCP appartient au reconciler IAM, ce qui est une dépendance à migrer en plus des templates du chart.

UTILISATEURS ET RÔLES

Services, intégrations et réconciliateurs IAM.

INTERFACES DANS LE CODE

Templates `_realm_clients.tpl`, `realm_clients/_api_client.tpl` et `_realm_client_scopes.tpl` ; `invariants.sh` pour clients non déclarés.

DONNÉES ET ÉTATS

Clients, secrets référencés, scopes, mappers et comptes techniques du realm.

DÉPENDANCES ET ÉCHANGES

IAM, APISIX et Kubernetes Secrets.

COMPOTEMENT CIBLE PROPOSÉ

IAM Rust provisionne des identités de service à portée explicite auprès d'un IdP sans JVM. Un token technique doit conserver une audience limitée et l'acteur lorsqu'il agit par délégation.

MIGRATION DE CETTE CAPACITÉ

Recenser les clients créés par chart et ceux créés par IAM, puis renouveler secrets et audiences. Remplacer les constructions d'URL Keycloak dans chaque client avant retrait.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une intégration continue de recevoir un token attendu par sa seule API. Un token sans audience gateway est refusé et une intégration ne peut échanger pour une audience non autorisée.

Références de l'analyse

[monorepo](#) · [services/keycloak/chart/templates/realm_clients/_api_client.tpl:60](#) Compte technique du client confidentiel.

[monorepo](#) · [services/keycloak/chart/files/realm-sync/invariants.sh:228](#) Audience et scope des clients non déclarés par le chart.

KC-04 Cible non qualifiée**Appliquer politiques de session, MFA et protections du login****COMPORTEMENT OBSERVÉ**

La définition du realm déclare politique de mot de passe, lifespans de tokens et sessions, rotation des refresh tokens, protections brute force et headers navigateur. Le flow Jupyterhub peut être conditionné par un rôle. Les invariants rendent le sous-flow MFA du navigateur obligatoire lorsque demandé, et ne relâchent pas un realm déjà renforcé. La définition enregistre les événements de connexion, échec, logout, mot de passe et MFA pour l'exploitation.

UTILISATEURS ET RÔLES

Utilisateurs et responsables sécurité.

INTERFACES DANS LE CODE

`_realm_definition.tpl` ; `_realm_auth_flows.tpl` ; `_realm_events.tpl` ; `invariants.sh` section MFA.

DONNÉES ET ÉTATS

Politiques et flows du realm, credentials MFA, sessions et événements Keycloak.

DÉPENDANCES ET ÉCHANGES

IAM pour les écrans d'administration ; Jupyterhub ; IdP.

COMPORTEMENT CIBLE PROPOSÉ

Traduire chaque politique dans un modèle de configuration IAM versionné, puis qualifier son effet avec le fournisseur sans JVM. Les facteurs et passkeys dont l'export n'est pas compatible nécessitent un réenrôlement explicite.

MIGRATION DE CETTE CAPACITÉ

Comparer les politiques réellement actives, y compris celles que le chart ne possède pas. Tester MFA, expiration, logout et anti-brute-force ; ne pas annoncer une portabilité automatique des credentials.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un compte soumis à MFA ne peut terminer le login avec le seul mot de passe. Un déploiement ne relâche pas la politique appliquée. Les bornes de session arrêtent les délégations associées.

Références de l'analyse

[monorepo](#) · [services/keycloak/chart/templates/_realm_definition.tpl:90](#) Politiques et paramètres déclarés du realm.

[monorepo](#) · [services/keycloak/chart/files/realm-sync/invariants.sh:268](#) Renforcement MFA à sens unique.

[monorepo](#) · [services/keycloak/chart/templates/_realm_auth_flows.tpl:9](#) Flow de login conditionné par rôle.

KC-05 Cible non qualifiée**Réconcilier le realm à chaque déploiement et détecter la dérive****COMPORTEMENT OBSERVÉ**

Le Job post-install/post-upgrade exécute wait, import, invariants puis realm-diff dans un ordre imposé par les init containers. Un échec bloque le Job et le déploiement. L'import met à jour les objets déclarés sans supprimer ceux laissés hors configuration. Les invariants traitent notamment profil admin-only, bootstrap, marqueurs de bienvenue, clients à retirer par nom et vérification des credentials. Le CronJob de drift utilise un client en lecture seule ; il ne couvre pas les mappings de rôles des comptes techniques ni le realm master.

UTILISATEURS ET RÔLES

Exploitation et automatisation de déploiement.

INTERFACES DANS LE CODE

Job realm-sync ; CronJob realm-drift ; scripts wait.sh, invariants.sh et realm-diff.sh.

DONNÉES ET ÉTATS

Définition Helm, objets effectifs du realm et résultats de Job ; configuration hors propriété du chart préservée.

DÉPENDANCES ET ÉCHANGES

Kubernetes, Helm, API Admin Keycloak et keycloak-config-cli JVM.

COMPORTEMENT CIBLE PROPOSÉ

Un reconciler Rust et des jobs bornés remplacent keycloak-config-cli, l'opérateur et les scripts dépendant de Keycloak. Chaque propriété gérée a une comparaison avant/après et une preuve de convergence ; les extensions clientes restent hors propriété explicite.

MIGRATION DE CETTE CAPACITÉ

Séparer objets possédés, objets seulement lus et exceptions clientes. Porter invariants et drift avant la bascule d'IdP ; supprimer ensuite toutes les images JVM de sync et d'exploitation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un écart dans une propriété gérée fait échouer la vérification. Un fournisseur fédéré hors propriété du chart reste intact. Deux applications successives de la même configuration produisent le même état.

Références de l'analyse

[monorepo · services/keycloak/chart/templates/realm-sync.yaml:12](#) Ordonnancement et blocage sur échec.

[monorepo · services/keycloak/chart/templates/realm-sync.yaml:101](#) Protection no-delete des types gérés.

[monorepo · services/keycloak/chart/templates/realm-drift.yaml:6](#) Limites du contrôle périodique en lecture seule.

KC-06 **Cible non qualifiée**

Personnaliser le login, les emails et les premiers accès**COMPORTEMENT OBSERVÉ**

L'image assemble un thème et un plugin JAR. Le realm choisit thème de login et d'email, langues, locale et configuration SMTP via une référence de coffre. Le listener welcome-email est branché aux événements. Lors de la première adoption par le sync, les utilisateurs existants reçoivent un marqueur qui évite un envoi de bienvenue collectif ; les nouvelles identités suivent ensuite le listener. Les invariants protègent aussi les attributs de politique et de bienvenue dans le profil utilisateur.

UTILISATEURS ET RÔLES

Utilisateurs, administrateurs de branding et exploitation.

INTERFACES DANS LE CODE

Dockerfile de l'image ; _realm_definition.tpl ; invariants.sh section welcome-email.

DONNÉES ET ÉTATS

Thèmes, locale, configuration SMTP, attributs de profil et marqueurs de bienvenue.

DÉPENDANCES ET ÉCHANGES

Thème et plugin partagés, SMTP, IAM et futur Notifications.

COMPORTEMENT CIBLE PROPOSÉ

Conserver les écrans en TypeScript ou dans un thème supporté par l'IdP sans JVM. Notifications Rust porte l'envoi métier de bienvenue avec déduplication ; le mécanisme de sécurité du mot de passe reste au fournisseur d'identité.

MIGRATION DE CETTE CAPACITÉ

Inventorier traductions et parcours email sans recopier les paramètres SMTP. Importer les marqueurs de bienvenue et qualifier liens, expiration et changements de langue.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Migrer des comptes existants n'envoie pas de bienvenue en masse. Un nouveau compte reçoit au plus un message métier et le lien de sécurité est valide uniquement dans le contexte attendu.

Références de l'analyse

[monorepo · services/keycloak/app/Dockerfile:3](#) Assemblage des extensions JAR.

[monorepo · services/keycloak/chart/templates/_realm_definition.tpl:81](#) Listener branché et thèmes du realm.

[monorepo · services/keycloak/chart/files/realm-sync/invariants.sh:151](#) Marquage initial des comptes déjà présents.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Configuration OIDC, identités de service, fédérations et politiques versionnées par IAM Rust.
- Réconciliation explicite et inventaire des propriétés gérées sans suppression générale du realm.

Plan de données proposé

- Émission et validation de tokens par un fournisseur sans JVM ; aucun accès direct aux tables ou blobs conféré par le seul login.

Découpage du core Rust

- Adaptateur IdP, réconciliateur de clients/politiques et mapping issuer/sub dans IAM.
- Événements de bienvenue idempotents confiés à Notifications. Le serveur OIDC peut être un tiers sans JVM qualifié.

Identité, autorisation et isolation

- Ne pas relâcher PKCE, MFA, restriction des audiences ou lifespans pour faciliter la migration.
- Ne pas fusionner les personnes par courriel ni supposer portables les sessions, refresh tokens et facteurs MFA.

Exploitation et reprise

- Le chart installe une CR Keycloak adossée à PostgreSQL et un Job de convergence bloquant.
- La sortie zéro JVM couvre serveur, opérateur, JAR, image de sync et outils de runbook. Sauvegarde, restauration et fonctionnement sans Internet restent à qualifier pour le remplaçant.

Séquence de migration

- Inventorier clients, fédérations, facteurs et claims utilisés.
- Qualifier un IdP sans JVM et le reconciler sur un environnement isolé.
- Importer IDs et credentials compatibles, imposer réauthentification ou réenrôlement pour le reste.
- Basculer les audiences et clients, mesurer révocation, puis retirer l'ensemble des dépendances Keycloak.

Écarts et décisions encore ouverts

- Aucun fournisseur sans JVM n'a été testé dans cette mission.
- Le chart ne suffit pas à établir les fédérations, facteurs et politiques effectifs de chaque cluster.
- Les plugins et thèmes partagés sont cités comme dépendances ; ce lot ne relit pas intégralement leur code.

Références de l'analyse

[monorepo](#) · [services/keycloak/chart/templates/keycloak.yaml:1](#) Déploiement par l'opérateur Keycloak.

[monorepo](#) · [services/keycloak/app/Dockerfile:1](#) Version et runtime de l'image actuelle.